1. Methodology Under Review

The project adopts a Three-Tier Layered Architecture. The high-level architecture consists of:

- Presentation Layer (Flutter)
- Business Logic Layer (Repository + Firebase Cloud Functions)
- Data Layer (Cloud Firestore) The four core scenarios (use cases) are:
- UC-01: User Registration and Login (covers FR-01, FR-02, FR-03)
- UC-02: Reporting a Stray Dog Location (covers FR-04, FR-10, FR-11)
- UC-03: Viewing the Map with Reports and Filtering (covers FR-05, FR-06, FR-07, FR-13)
- UC-04: Admin Removing a False Report (covers FR-08, FR-09)

2. Technological Context PawSafe is built for a safety-critical, real-time mobile application scenario requiring continuous data synchronization. Key requirements from the Requirements Document include:

- FR-04: GPS-tagged report submission with optional photo (max 5 MB) and description (max 200 characters)
- FR-05: Live map refresh every 30 seconds with color-coded markers
- FR-10: Rate limiting — maximum 5 report submissions per user per hour, enforced via Firebase Cloud Functions
- FR-11: Push notifications for reports within a user-configured radius (FCM)
- NFR-01: Map load and report submission confirmation both within 3 seconds on 4G
- NFR-02: All data transmitted over TLS; Firebase Security Rules enforce role separation Design patterns chosen:
- Strategy Pattern (GoF Behavioural) — for report filtering algorithms (HourFilter, SixHourFilter, DayFilter, AllFilter), allowing runtime algorithm swapping without modifying client code
- Singleton Pattern (GoF Creational) — for LocationService, ensuring a single GPS hardware session is shared across screens to prevent battery drain

3. Summary of the Method PawSafe combines Firebase's serverless infrastructure with Flutter's cross-platform rendering. Firebase handles authentication, data sync, and push notifications. Reactive state is managed through Riverpod providers: UI components observe streams and rebuild automatically. Filtering logic is decoupled via the Strategy Pattern. GPS hardware access is centralized through the LocationService Singleton. Admin moderation uses GoRouter guards and Firebase Security Rules for role-based access.

4. System Evaluation

The system is thoroughly defined, demonstrating strong architectural alignment between the functional requirements and the technical design. The Design Document provides:

- **Roles**: User (standard authenticated user) and Admin (role-enforced via Firestore and GoRouter guards).
- **Processes**: Four core use cases (UC-01 through UC-04) with explicit linkages to Functional Requirements (FR).
- **UML Coverage**: Comprehensive documentation including Component Diagram (Figure 1), Class Diagram with FilterStrategy hierarchy (Figure 2), Sequence Diagrams for report submission and map filtering (Figures 3–4), Deployment Diagram (Figure 5), and pattern-specific Class Diagrams for Strategy and Singleton (Figures 6–7).
- **Separation of Concerns**: Separation is strictly enforced across the three-tier architecture; UI screens do not contain business logic, repositories do not reference UI widgets, and Cloud Functions handle backend enforcement (such as rate limiting and FCM dispatch) independently of the mobile client.

5. Assumption Validity

Valid Assumptions

- **GPS Permissions**: Users will grant the application high-accuracy GPS access to capture coordinates automatically for reports (FR-04).
- **Cross-Platform Fluidity**: The Flutter rendering engine will maintain 60fps performance for smooth map interactions on both Android and iOS (NFR-01).
- **Role-Based Security**: Firebase Security Rules and GoRouter guards will effectively prevent regular users from accessing admin-only deletion endpoints (FR-08).

Invalid Assumptions

- **Sufficient Rate Limit**: The assumption that a limit of 5 reports per hour per user (FR-10) is sufficient for safety. This becomes invalid in fast paced or emergency scenarios.
- **Exclusively Manual Moderation**: The assumption that manual admin deletion (FR-09) is the only necessary mechanism for data quality. This fails to account for the rapid staleness of real-time safety data in a high-volume environment.
- **Sufficient Information from New Markers**: The assumption that safety is best served by creating a new marker for every sighting. This is invalidated by the fact that re-submissions consume the user's rate limit and create map clutter instead of updating existing moving hazards.

6. Identified Weaknesses

1. Static Rate Limiting Risk (FR-10): The limit of 5 submissions per hour introduces a safety bottleneck. The 12-minute average interval is too slow for fast-moving hazards.
2. Scalability of Moderation: Reliance on manual admin moderation for removing false reports may not scale as the user base increases.

7. Missing Mechanisms

1. Community-Driven Marker Validation: A mechanism to 'confirm' existing markers instead of creating duplicates to keep data fresh without hitting rate limits.
2. Automatic Data Expiration Policy: A system to automatically archive or fade markers after a specific timeframe (e.g., 2 hours) to prevent false security.

8. Scenario Evaluation

Realistic Scenario: A user spots a pack of dogs moving rapidly through a neighbourhood. They submit the first report. For example, two minutes later, the pack turns into a crowded group. Because of the 12-minute average wait time imposed by the rate limit (5 reports per hour), the user cannot update the location. The app continues to show the dogs at the previous street, creating a false sense of security for the users.

9. Improved Assessment

The Strategy Pattern for filtering is an excellent choice for flexibility. However, for safety improvements, the filtering logic should prioritize recency even more aggressively and perhaps introduce an Emergency/ High Mobility mode that temporarily lifts the rate limitation for the verified users. The system should implement a dynamic rate-limiting override or an Emergency Mode to ensure that the community receives immediate and relevant safety alerts.

10. Implementation Assessment

The prototype plan effectively covers core features. However, The 5- reports- per hour limit blocks the system from sending live alerts. Because of the strict report limit, notifications will show old data. If a dog is moving fast, the app can't send new alerts, so users will not know the actual danger in time**.**

11. Overall Strengths

1) **The Choice of Architecture**: The project uses Layered Architecture, which creates a professional and clean separation between the UI, business logic, and database. This makes the codebase much easier to maintain and allows different members to work on separate modules.
2) **High Performance:** By choosing Flutter over other frameworks, the app benefits from a high-performance rendering engine. This is especially important for smooth map interactions and maintaining a consistent look across both Android and İOS.
3) **Detailed Technical Documentation:** The design includes clear UML diagrams that map directly to the functional requirements. This level of detail shows a deep understanding of the software development lifecycle process.

12. Overall Limitations

Rate Limitation: The strictness of the rate limit (5 /per hour) contradicts the system'S primary objective of ensuring neighbourhood safety in dynamic environments.

Platform Dependancy: Complete reliance on Google/Firebase makes the system vulnerable to vendor lock-in.

13. Recommendation

Tasarımı kabul eder misiniz: ☐ Accept ☒ **Weak Accept** ☐ Borderline ☐ Weak Reject ☐ Reject

Recommendations:

1) **Implement a Dynamic Emergency Mode:**
   The system should allow users to rate more than 5 times per hour when a high mobility threat is detected. If the rates are strictly limited, notifications will fail to provide the real-time safety updates.

2) **Enable Marker Updates:**

The system should allow users to confirm or update an existing marker’s position instead of creating a brand new report. This would reduce map clutter while keeping location data fresh without hitting the rate limit.

14. Justification

The design of PawSafe demonstrates high technical maturity and follows professional software engineering standards. However, the 5 reports-per-hour limit creates a potential bottleneck for reporting fast-moving threats. An Emergency Mode update would ensure that users can provide fast updates when a pack of dogs changes location rapidly. Also, enabling updates for markers like (still here or gone buttons) would complement the admin deletion process, making the moderation faster and more reliable through the community.